

FULL-STACK FRONTEND TRACK · WEEK 3 · DAY 7

Lab Task

Fundamentals & OOP — converting the Inventory Utility to real TypeScript

Hands-on Practice — covers a combined two-day agenda

Extended session — 8 topics

Contents

- 1. Overview & Instructions
- 2. Warmups — One Per Concept
- 3. Applied Project — Typing the Inventory Utility
- 4. Self-Check Checklist

1. Overview & Instructions

Today covers everything from this session's lecture — fundamentals through full OOP. The applied project takes your actual Week 2 Inventory Utility (the API-backed version from Day 6) and rebuilds it in TypeScript from the ground up: real interfaces, real access modifiers, a generic repository, and an abstract base class.

Rules for This Task

Do

Write every file as `.ts` , with `"strict": true` in your `tsconfig.json` from the start. Compile with `tsc` (or `tsc --watch`) and confirm zero errors before considering any part done. Let inference handle obvious variable types; annotate every function's parameters and return type explicitly.

Don't

No `any` anywhere in your code — if a type genuinely isn't known yet, use `unknown` and narrow it. Don't import a `.ts` file by its `.ts` name — import paths reference the compiled `.js` output, exactly as shown in today's demo.

Deliverables

File	Purpose
warmups.ts	Section 2
tsconfig.json	Project-wide compiler settings, strict mode on
src/product.ts	Interfaces, the abstract base class, and the generic repository
src/api.ts	Your Day 6 <code>api.js</code> , retyped
inventory.html + src/inventory.ts	The rebuilt page

2. Warmups — One Per Concept

warmups.ts

Compile this file on its own (`tsc warmups.ts`) and confirm it compiles with zero errors before moving on, except where a task explicitly asks you to demonstrate an error.

2.1 — Setup

- Create a `tsconfig.json` using `tsc --init`, then edit it down to just: `target`, `module`, `strict`, `outDir`, `rootDir`

2.2 — Type Annotations & Inference

- Declare a typed array of at least 4 order totals (`number[]`), and a tuple representing one product's `[name, price]`
- Declare one variable with **no** annotation and confirm (by hovering in your editor, or trying to assign a wrong type) that TypeScript still correctly inferred it
- Write one line using `unknown` for a value of uncertain type, and show the `typeof` check required before you can use it

2.3 — Union & Intersection Types

- Write a function accepting a parameter typed `string | number`, and use narrowing to handle both cases differently
- Define two small object types and combine them with `&` into a third type; create one value that satisfies it

2.4 — Typed Functions

- Write one function with a required parameter, one optional parameter (`?`), and an explicit return type
- Write a second function with a default parameter value

2.5 — Classes & Access Modifiers

- Write a class with at least one `private` field, one `readonly` field, and one field declared using the constructor-parameter-property shorthand

2.6 — Interfaces & Structural Typing

Requirement

Define an interface, then create a plain object literal that satisfies it **without** ever writing `implements` and with at least one extra property the interface doesn't mention. Pass it to a function typed to accept that interface, and confirm it compiles.

2.7 — Inheritance & Abstract Classes

- Write an abstract class with one abstract method and one concrete method that calls it
- Write a subclass implementing the abstract method, instantiate it, and call the concrete method

2.8 — Generics

- Write a generic function that works on an array of any type and returns its last element
- Write a generic class with at least one constraint (`T extends ...`)

3. Applied Project — Typing the Inventory Utility

1. Project Setup

- A `tsconfig.json` with `strict: true`, `rootDir: "./src"`, `outDir: "./dist"`
- All source files live in `src/` as `.ts`; `inventory.html` loads the compiled output from `dist/` with `<script type="module">`

2. Define the Core Types

`src/product.ts`

- A `Product` interface: `id`, `name`, `price`, `stock`, `category`
- An **abstract** `Entity` base class with a `readonly id: number` field and an abstract `describe(): string` method
- A `ProductEntity` class that `extends Entity`, implements `Product`, and implements `describe()`
- A generic `Repository<T extends { id: number }>` class with `add`, `getAll`, and `findById` methods

3. Retype the API Layer

`src/api.ts`

Requirement

Convert your Day 6 `api.js` so every function has typed parameters and a typed return (e.g. `getProducts(): Promise<Product[]>`). The actual `fetch` logic doesn't change — only the type annotations are new.

4. Rebuild the Inventory Page

`src/inventory.ts`

- On load, fetch products and load them into a `Repository<Product>`
- A typed function computing total inventory value, and one computing low-stock count — both with explicit parameter and return types
- A union-typed lookup function accepting either a numeric id or a product name string, used somewhere in the page (e.g. a search box that accepts either)
- Rendering, Create, Edit, and Delete all continue to work exactly as they did in Day 6 — nothing about the actual behavior changes, only that everything is now type-checked

5. Prove `strictNullChecks` Is Doing Something

Requirement

Somewhere in `inventory.ts`, call `document.getElementById(...)` and show the null check TypeScript forces before you can use the result — the same pattern from today's demo.

4. Self-Check Checklist

	Requirement
☐	<code>tsc</code> compiles the entire project with zero errors, strict mode on
☐	No <code>any</code> appears anywhere in the project
☐	<code>Product</code> interface, abstract <code>Entity</code> , <code>ProductEntity</code> , and generic <code>Repository<T></code> all exist and are used together correctly
☐	Every function in <code>api.ts</code> has explicit parameter and return types
☐	At least one union-typed function with real narrowing logic is used in the running page
☐	A structural-typing example (Warmup 2.6) exists and compiles without <code>implements</code>
☐	All CRUD operations (Create/Read/Update/Delete) still work exactly as they did in Day 6
☐	A null-check on a <code>document.getElementById</code> result appears and is required for the file to compile

Looking ahead

Day 8 adds enums, utility types, and a look at real-world TypeScript configuration and tooling — then Week 4 (Angular) is where everything from this week becomes the language the framework itself is written in.